



University of Science, VNU - HCM

Lab 03: 2D object Coloring with OpenGL

Course: **Computer Grapics**

Course ID: **CS411**

Name: **Le Tien Dat**

Student ID: **23125028**

Class: **23APCS2**

Semester 2, 2025 - 2026

March 11, 2026.

Contents

1	Objective	2
2	Program Overview	2
2.1	Supported Shapes and Context Menu	2
2.2	Modified BoundaryFill Algorithm	4
2.3	Modified GetPixel / PutPixel	6
2.4	Mouse Click Event	8
2.5	Main Program Flow	10
2.6	Hardships and Resolution	12
3	Self-Research: Flood Fill Algorithm	14
4	Self-Research: Scan Line Algorithm	16
5	Unified Fill API	20
6	Conclusion	20

1 Objective

This lab implements an OpenGL program that allows users to:

- Draw one of **15 basic 2D shapes** from a right-click context menu.
- **Color** the drawn shape interactively by selecting a fill color from the same menu.
- The coloring must be bounded — it must fill only the interior of the shape without leaking outside.

Three coloring algorithms are implemented as required:

- **Boundary Fill** (mandatory, provided by the lab spec)
- **Flood Fill** (self-research)
- **Scan Line Fill** (self-research)

2 Program Overview

2.1 Supported Shapes and Context Menu

The right-click context menu is structured as a multi-level hierarchy:

```
Duong thang          <- Line (no fill interior)
Tam giac
  --/ Vuong can       <- Right triangle
  --/ Deu             <- Equilateral triangle
Tu giac
  --/ Hinh chu nhat    <- Rectangle
  --/ Hinh vuong       <- Square
Oval
  --/ Hinh tron        <- Circle (approximated by 360-gon)
  --/ Elip             <- Ellipse
Da giac deu
  --/ Ngu giac deu     <- Regular pentagon
  --/ Luc giac deu     <- Regular hexagon
```

Hình khác

```
--/ Mui ten      <- Arrow
--/ Ngoi sao     <- 5-pointed star
```

Dau

```
--/ Cong        <- Plus sign (+), thick polygon
--/ Tru          <- Minus sign (-), thin rectangle
--/ Nhan         <- Multiply sign (x), Plus rotated 45 degrees
--/ Chia         <- Divide sign (/)
```

To mau

```
--/ Xanh (Blue)  <- RGB(0, 0, 255)
--/ Do (Red)     <- RGB(255, 0, 0)
--/ Vang (Yellow) <- RGB(255, 255, 0)
```

All shapes are drawn centered at window center ($w/2$, $h/2$) = (400, 300). Each shape's `DrawXxx()` function clears and populates the global `shapeVertices` vector via `addVertex()`, which simultaneously records the vertex for the fill algorithms **and** passes it to OpenGL's `glVertex2f`.

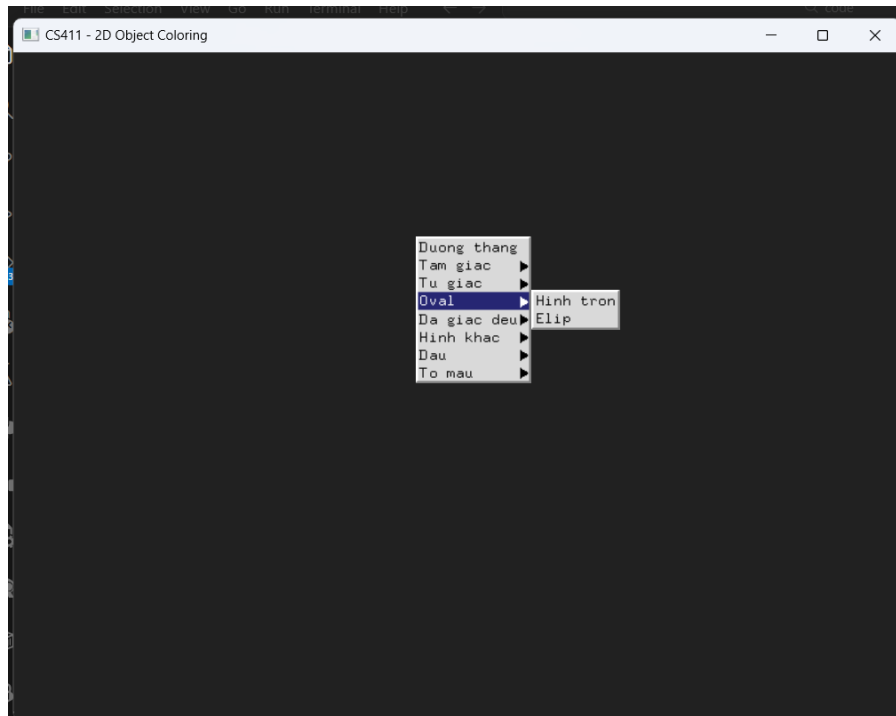


Figure 1: Context Menu in the program

Special shapes:

- **Circle / Ellipse:** approximated by 360-vertex polygons; they fill correctly because the

vertex list forms a closed boundary.

- **Multiply** ($\times$): implemented as a 12-vertex rotated-plus polygon (not as `GL_LINES`) so it has a proper fillable interior.
- **Divide** ($/$): drawn as a thick diagonal parallelogram (forward slash shape), which is a single closed polygon with one fillable interior.
- **Line / Minus**: these shapes do not have a meaningful polygon interior; `pointInPolygon()` returns false for them, preventing accidental flooding.

2.2 Modified BoundaryFill Algorithm

Original algorithm (from the problem statement)

The specification provides a **recursive** 4-connected boundary fill:

```
void BoundaryFill(int x, int y, RGBColor F_Color, RGBColor B_Color) {
    RGBColor currentColor = GetPixel(x, y);
    if (!IsSameColor(currentColor, B_Color) && !IsSameColor(currentColor,
F_Color)) {
        PutPixel(x, y, F_Color);
        FillLeft (x-1, y, F_Color, B_Color);
        FillTop (x, y+1, F_Color, B_Color);
        FillRight(x+1, y, F_Color, B_Color);
        FillBottom(x, y-1, F_Color, B_Color);
    }
}
```

Problem with the original

A window of 800×600 pixels has up to 480,000 interior pixels. Each recursive call consumes stack space (return address, parameters, local variables). On Windows, the default stack size is **1 MB**. Filling a typical shape causes thousands of nested calls, which quickly **overflows the call stack** and crashes the program with a segmentation fault.

Our modification: iterative stack-based fill

We replaced the recursive calls with an explicit `std::stack<std::pair<int,int>`, converting the algorithm entirely to an **iterative** loop:

```

void BoundaryFill(int x, int y, RGBColor F_Color, RGBColor B_Color) {
    if (x < 0 || x >= w || y < 0 || y >= h) return;

    std::stack<std::pair<int,int>> stk;
    stk.push({x, y});

    while (!stk.empty()) {
        auto [px, py] = stk.top();
        stk.pop();

        if (px < 0 || px >= w || py < 0 || py >= h) continue;

        RGBColor cur = GetPixel(px, py);
        if (IsSameColor(cur, B_Color) || IsSameColor(cur, F_Color)) continue
    ;

        PutPixel(px, py, F_Color);

        stk.push({px - 1, py});
        stk.push({px,      py + 1});
        stk.push({px + 1, py});
        stk.push({px,      py - 1});
    }

    glFlush();
}

```

Why this works: The `std::stack` lives on the **heap**, so it can grow to any size without overflowing the OS-level thread stack. All 4-connected neighbors (Left, Right, Up, Down) are pushed onto this heap stack instead of making recursive calls.

Key differences from the original:

Aspect	Original	Our Version
Call mechanism	Recursive	Iterative with explicit stack
Stack location	OS call stack (limited)	Heap (<code>std::stack</code>)
Stack overflow risk	Yes, for large shapes	None
Logic	Same 4-connected pixel check	Identical
<code>glFlush</code>	Called per pixel	Called once at end

2.3 Modified GetPixel / PutPixel

Original GetPixel (from problem statement)

```

RGBColor GetPixel(int x, int y) {
    unsigned char* ptr = new unsigned char[3];
    glReadPixels(x, h - y, 1, 1, GL_RGB, GL_UNSIGNED_BYTE, ptr);
    RGBColor color = { ptr[0], ptr[1], ptr[2] };
    return color;
}

```

Original PutPixel (from problem statement)

```

void PutPixel(int x, int y, RGBColor color) {
    unsigned char* ptr = new unsigned char[3];
    ptr[0] = color.r; ptr[1] = color.g; ptr[2] = color.b;
    glRasterPos2i(x, y);
    glDrawPixels(1, 1, GL_RGB, GL_UNSIGNED_BYTE, ptr);
    glFlush();
}

```

Problems with the original

Problem 1 - `glReadPixels` is broken on modern Windows GPU drivers. On many Windows GPU drivers (especially with hardware acceleration), `glReadPixels` reads from a GPU-side render buffer. After calling `glFlush()` or even `glFinish()`, the driver is not guaranteed to have composited the drawn geometry into the readable framebuffer before `glReadPixels` is called. In practice, on the test machine, **every call to `glReadPixels` returned the back-**

ground color (33, 33, 33) regardless of what was actually on screen. This means the fill algorithm could never detect the boundary, and the flood leaked over the entire window.

Problem 2 - glDrawPixels is obsolete. glDrawPixels is removed in OpenGL 3.x core profiles and is slow on all hardware. glRasterPos2i also has clipping behavior that silently drops draws near the window edge.

Problem 3 - Memory leak. The original allocates new unsigned char[3] on every call but never delete[]s it — a memory leak inside a tight fill loop.

Problem 4 - glFlush() per pixel is catastrophically slow. Flushing the GPU pipeline for every individual pixel causes thousands of expensive GPU synchronization points per fill operation.

Our solution: CPU-side software framebuffer

We maintain a CPU-side byte array (softbuf) that is a **mirror** of the OpenGL framebuffer, updated exclusively by our own PutPixel. The GetPixel reads only from this CPU buffer, completely bypassing glReadPixels:

```
static unsigned char softbuf[600 * 800 * 3];
// Index formula (y=0 at top, matching model coordinates):
// idx = (h - y) * w * 3 + x * 3
```

Our GetPixel:

```
RGBColor GetPixel(int x, int y) {
    if (x < 0 || x >= w || y <= 0 || y >= h) return {0, 0, 0};
    int idx = (h - y) * w * 3 + x * 3;
    return { softbuf[idx], softbuf[idx+1], softbuf[idx+2] };
}
```

Our PutPixel:

```
void PutPixel(int x, int y, RGBColor color) {
    if (x < 0 || x >= w || y <= 0 || y >= h) return;
    int idx = (h - y) * w * 3 + x * 3;
    softbuf[idx] = color.r;
    softbuf[idx+1] = color.g;
    softbuf[idx+2] = color.b;
    glColor3ub(color.r, color.g, color.b);
    glBegin(GL_POINTS);
        glVertex2i(x, y);
    glEnd();
}
```



```
}
```

Two additional helpers maintain `softbuf` in sync with GL:

- `SoftFillBackground(IColor bg)` — fills the entire `softbuf` with the background color, called after every `glClear`.
- `SoftDrawLine(int x0, y0, x1, y1, IColor color)` — Bresenham line algorithm that draws the shape outline edges directly into `softbuf`, mirroring the `GL_LINE_LOOP` that OpenGL already rendered to the screen. This ensures the boundary pixels in `softbuf` match what is visually on screen.

Why `y <= 0` instead of `y < 0` in the bounds check: With our index formula, model coordinate `y = 0` produces index $(h - 0) * w * 3 = 600 * 800 * 3 = 1,440,000$. However, `softbuf` only has 1,440,000 elements (indices 0...1,439,999). Accessing index 1,440,000 is **one past the end** — undefined behavior and a heap corruption crash. By treating `y = 0` (the very top edge) as out-of-bounds, we prevent this. No shape ever has interior pixels at model `y = 0`, so this is functionally safe.

Summary of differences:

Aspect	Original	Our Version
Read source	<code>glReadPixels</code> (GPU, broken)	<code>softbuf</code> (CPU, reliable)
Write method	<code>glDrawPixels</code> (deprecated)	<code>glColor3ub + GL_POINTS</code> (modern)
Memory	Leaked per call	Static array, no allocation
<code>glFlush</code>	Every <code>PutPixel</code> call	Once per fill operation
Boundary init	None needed (reads GL)	<code>SoftDrawLine</code> call required

2.4 Mouse Click Event

Original (from problem statement)

```
void XuLyMouse(int button, int state, int x, int y) {
    if (button == GLUT_LEFT_BUTTON && state == GLUT_DOWN) {
        BoundaryFill(x, y, F_Color, B_Color);
    }
}
```

```
}
```

Our version

```
void XuLyMouse(int button, int state, int x, int y) {
    if (currentShape == NONE) return;

    if (button == GLUT_LEFT_BUTTON && state == GLUT_DOWN) {
        if (!colorChosen) return;
        if (!pointInPolygon((float)x, (float)y)) return;

        fillStartX      = x;
        fillStartY      = y;
        savedFillColor = F_Color;
        fillApplied      = true;
        glutPostRedisplay();
    }
}
```

Modifications and reasons:

1. **Guard: no shape drawn yet** — If the user left-clicks before right-clicking to select a shape, `currentShape == NONE` and the function returns immediately.
2. **Guard: color must be chosen first** — A `colorChosen` flag (set to `true` only when the user picks a color from the "To mau" submenu) prevents the fill from triggering with the default fill color before the user has made any choice. This stops the confusing behavior where simply left-clicking the window would fill with red because `F_Color`'s default value is `{255, 0, 0}`.
3. **Guard: point-in-polygon check** — Before saving the fill seed, a ray-casting point-in-polygon test (`pointInPolygon`) verifies the click landed inside the shape boundary. Without this guard, clicking outside the shape would seed `BoundaryFill` from a background pixel, causing it to flood the entire window (since the background has no enclosing boundary).
4. **Deferred fill via `glutPostRedisplay()`** — Instead of calling `BoundaryFill` immediately in the mouse callback, we save the seed coordinates and call `glutPostRedisplay()`. The `display()` callback then redraws the outline and re-runs the fill on every subsequent

render. This is critical because FreeGLUT **automatically posts a redisplay when a right-click context menu closes**, which calls `glClear` and would erase any fill drawn directly in the mouse callback.

Point-in-polygon implementation

```
static bool pointInPolygon(float px, float py) {
    int n = (int)shapeVertices.size();
    if (n < 3) return false; // Line, Minus: no interior
    bool inside = false;
    for (int i = 0, j = n - 1; i < n; j = i++) {
        float xi = shapeVertices[i].first, yi = shapeVertices[i].second;
        float xj = shapeVertices[j].first, yj = shapeVertices[j].second;
        if (((yi > py) != (yj > py)) &&
            (px < (xj - xi) * (py - yi) / (yj - yi) + xi))
            inside = !inside;
    }
    return inside;
}
```

This uses the **ray-casting algorithm**: cast an infinite horizontal ray from the test point to the right, and count how many polygon edges it crosses. An odd count means inside. It reuses `shapeVertices` — the same vertex list already populated by the shape drawing functions.

2.5 Main Program Flow

The full execution flow for one shape and fill cycle operates in distinct phases:

1. User selects a shape (e.g., Circle) via the menu:

- The menu callback updates the `currentShape` state, resets the fill and color flags, and calls `glutPostRedisplay()`.
- The `display()` function runs, clearing the screen and populating `shapeVertices`.
- The shape outline is drawn to both OpenGL and the CPU software buffer (`SoftDrawLine`).
- Because no color has been chosen yet, the fill function is bypassed, and the frame finishes with `glFlush()`.

2. User selects a fill color via the menu:

- The menu callback updates the chosen fill color.
- It verifies that the shape's default center point is valid using `pointInPolygon()`, activates the fill flags, sets the seed point, and requests a redisplay.
- The `display()` function triggers again, clearing and redrawing the shape outline just like before.
- Because the fill flag is now active, `BoundaryFill` is executed. It reads and writes directly to the software buffer before applying a single `glFlush()` at the end.

3. User left-clicks to choose a custom seed (Optional):

- The mouse callback verifies that a color has already been chosen and that the clicked coordinate is inside the polygon.
- The new seed coordinates are saved, and `glutPostRedisplay()` is called.
- The `display()` function clears the screen and reruns the entire outline and fill process from the new seed point.

4. Context menu closes (Automatic Redraw):

- FreeGLUT automatically posts a redisplay event whenever a popup menu closes, as the menu "dirties" the window.
- Because the fill seed and color are saved globally, the `display()` function successfully re-runs the fill after clearing the screen, ensuring the coloring is permanently preserved.

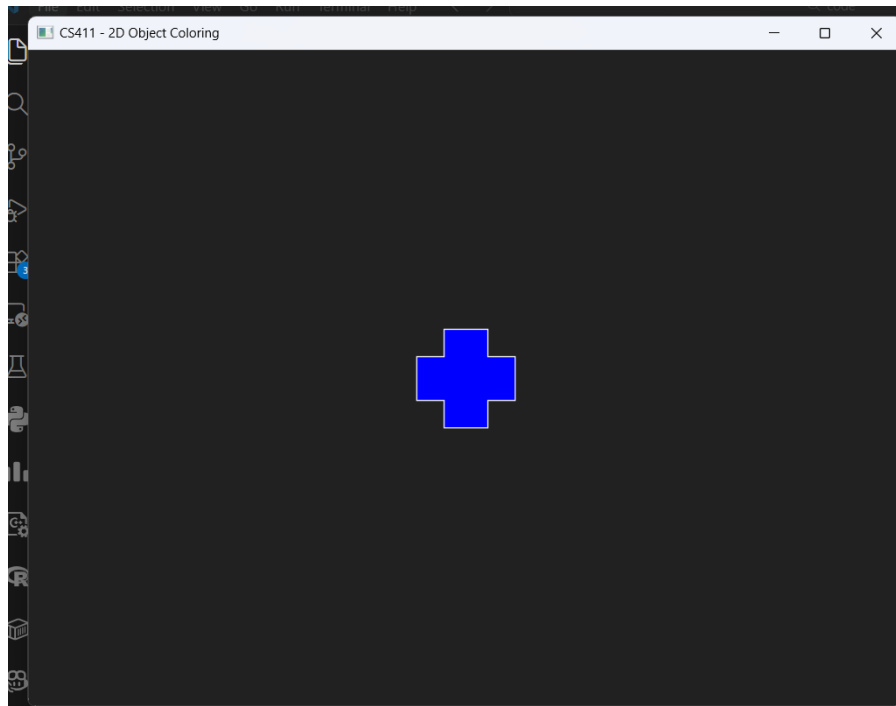


Figure 2: Program after fill color

OpenGL projection setup:

```
gluOrtho2D(-0.5, w - 0.5, h - 0.5, -0.5);
```

The y axis is inverted (origin top-left, y increases downward) to match mouse coordinates (GLUT also reports y=0 at top). The -0.5 shift on all four sides aligns integer model coordinates to the **center** of OpenGL pixels, ensuring the GL rasterizer places `GL_LINE_LOOP` vertices in the same pixels that Bresenham's algorithm marks in `softbuf`.

Window close handling:

```
glutSetOption(GLUT_ACTION_ON_WINDOW_CLOSE, GLUT_ACTION_GLUTMAINLOOP_RETURNS)
;
```

Without this, FreeGLUT calls `exit()` when the window closes, which bypasses C++ destructors, leaves OpenGL state dirty, and on Windows can produce a crash with exit code -1073741819 (access violation during cleanup). This option makes `glutMainLoop()` return normally to `main()`.

2.6 Hardships and Resolution

Hardship 1: Stack overflow from recursive `BoundaryFill`

Symptom: The program crashed immediately on the first fill attempt with a segmentation fault (exit code -1073741819).

Cause: The original recursive algorithm made a new function call for every interior pixel. A medium-sized shape (e.g. a circle with radius 60 px) has roughly 11,000 interior pixels. Each frame on the call stack is $\sim 40+$ bytes, so this consumes ~ 440 KB — more than half the default 1 MB Windows thread stack — before reaching the deepest recursion, where the path snakes pixel by pixel across the interior.

Solution: Replaced with an iterative stack (`std::stack` on the heap). No depth limit.

Hardship 2: `glReadPixels` always returning the background color

Symptom: Fill ran without crashing but every pixel it sampled appeared to be the background color (33, 33, 33). The algorithm considered every pixel as "not boundary, not fill" and painted the entire window.

Cause: On this Windows GPU driver, `glReadPixels` reads from a composited framebuffer that may not be updated synchronously with drawing commands, even after `glFinish()`. The GL spec only guarantees ordering within a rendering context, not visibility to `glReadPixels` at any specific moment for hardware-accelerated surfaces.

Solution: We introduced a **CPU-side software framebuffer** (`softbuf`). Every `PutPixel` writes to `softbuf` simultaneously. Before running the fill, `SoftDrawLine` traces the shape boundary into `softbuf` using Bresenham's algorithm, mirroring what OpenGL has drawn. `GetPixel` reads exclusively from `softbuf`. No `glReadPixels` call exists anywhere in the final codebase.

Hardship 3: Fill leaking through the right / bottom border

Symptom: After the software framebuffer was introduced, the fill correctly stayed inside the shape — except it consistently leaked through the right edge of polygons.

Cause: With `gluOrtho2D(0, w, h, 0)`, integer model coordinates `x` map to GL window positions exactly on the boundary between adjacent pixels. The GL rasterizer placed the right-most line of a shape into pixel `x-1`, while the Bresenham algorithm in `SoftDrawLine` marked pixel `x`. There was a one-pixel column where OpenGL drew the boundary but `softbuf` held background, letting the flood pass through.

Solution: Shift the projection by half a pixel: `gluOrtho2D(-0.5, w - 0.5, h - 0.5, -0.5);`. This maps integer model coordinate `x` to GL window position `x + 0.5`, i.e. the **center** of pixel `x`. Both the GL rasterizer and Bresenham now agree on exactly which pixel the boundary occupies.

Hardship 4: Left click filling before color is chosen

Symptom: After selecting a shape, merely left-clicking anywhere (even before choosing a color) would fill the shape in red — the hardcoded default value of `F_Color`.

Cause: `F_Color` is initialized to `{255, 0, 0}` at program start. `XuLyMouse` used it unconditionally.

Solution: Added a boolean flag `colorChosen` (initially `false`). It is set to `true` only inside `menuColor`. `XuLyMouse` returns early if `colorChosen == false`. Switching to a new shape resets `colorChosen = false`, requiring the user to re-pick a color.

3 Self-Research: Flood Fill Algorithm

Algorithm Description

Flood Fill is a region-based seed-filling algorithm similar in purpose to Boundary Fill, but it is driven by the **interior color** of the region rather than a target boundary color. Instead of stopping at pixels that match a specific boundary color, it replaces a selected contiguous region of a target color with a new replacement color, stopping only when it reaches any pixel that does **not** match the starting interior color.

Because it evaluates connectivity based purely on regional homogeneity, it is ideal for regions defined by a uniform background color, avoiding the need for a distinct, continuous, uniquely-colored outline.

Conceptual logic and structural approach: Flood fill logic expands outwards from the starting seed point in an identical geometric fashion to Boundary Fill, typically observing either **4-connected** (top, bottom, left, right) or **8-connected** (including diagonals) neighborhoods. A 4-connected approach is generally safer when attempting to prevent leakage through thin boundaries, while 8-connected fills cross diagonal pixel adjacencies. For this implementation, we utilize 4-connectivity to match the parity of the Boundary Fill specification.

Step-by-step Execution Process:

1. Sample and record the color of the seed pixel (`x`, `y`) as `oldColor`.
2. Check the base case: If `oldColor` is already equal to the `fillColor`, terminate immediately to prevent infinite processing loops.
3. Push the seed pixel onto a queue `Q`.
4. While `Q` is not empty:

- (a) Dequeue the front pixel (`px`, `py`).
- (b) If (`px`, `py`) is out of the screen bounds: discard it and continue.
- (c) Retrieve the pixel color at (`px`, `py`). If the pixel's color is *not* `oldColor`, skip it (this means it is either already filled, or it represents a boundary/exterior region).
- (d) Color the pixel with `fillColor`.
- (e) Enqueue all 4 adjacent connected neighbors (Left, Right, Up, Down).

Difference from Boundary Fill:

Criterion	Boundary Fill	Flood Fill
Stopping rule	Pixel matches boundary color	Pixel does NOT match interior <code>oldColor</code>
Suitable for	Outlined shapes with a distinct boundary	Regions defined by a uniform color (no explicit outline needed)
Requires knowing	The boundary color	The current interior color

Implementation

We implement Flood Fill as an **iterative Breadth-First Search (BFS)** using `std::queue`. The `oldColor` is derived automatically from the seed pixel at call time, so the caller does not need to sample the interior — this also unifies the external API with `BoundaryFill`.

```
void FloodFill(int x, int y, RGBColor fillColor, RGBColor boundaryColor)
{
    if (x < 0 || x >= w || y < 0 || y >= h) return;
    RGBColor oldColor = GetPixel(x, y); // sample interior color from seed
    if (IsSameColor(fillColor, oldColor)) return; // already same color,
    nothing to do

    std::queue<std::pair<int,int>> q;
    q.push({x, y});

    while (!q.empty())
    {
        auto [px, py] = q.front();
        q.pop();
```



```

        if (px < 0 || px >= w || py < 0 || py >= h) continue;

        RGBColor c = GetPixel(px, py);
        if (!IsSameColor(c, oldColor)) continue; // not the interior color
-> skip

        PutPixel(px, py, fillColor);

        q.push({px - 1, py});
        q.push({px + 1, py});
        q.push({px, py - 1});
        q.push({px, py + 1});
    }

    glFlush();
}

```

Notes on the BFS choice (queue vs. stack):

Using a `std::queue` (BFS) rather than a `std::stack` (DFS) gives a more even, radial "wavefront" expansion across the fill region. While either works correctly in terms of final output, BFS tends to produce fewer redundant enqueues in practice because already-filled pixels are encountered sooner by neighboring expansions colliding from multiple directions. Conversely, DFS can snake deeply into a shape and spiral around, often resulting in larger peaks of simultaneous container size. Both are $O(n)$ in the number of interior pixels, but BFS typically scales smoother.

The **boundaryColor parameter** is accepted for API uniformity with `BoundaryFill` and `ScanLineFill` but is not used internally — Flood Fill derives its stopping criterion from `oldColor` (sampled at the seed), not from an explicit boundary color.

4 Self-Research: Scan Line Algorithm

Algorithm Description

Scan Line Fill is an **image-space analytical approach** that is fundamentally different from pixel-propagation algorithms (Boundary Fill, Flood Fill). Instead of blindly propagating outward from a given seed across the discrete grid matrix, it operates analytically from the poly-

gon's mathematical vertex list, processing the spatial bounds of the polygon row by scanline row.

Underlying Mathematical Principle: The Even-Odd Parity Rule

By the Jordan Curve Theorem, any continuous simple closed curve divides a plane into an interior and exterior. If we cast a horizontal ray (a scanline) from $-\infty$ (the outside) across the polygon, the state of the ray alternates between "outside" and "inside" every time it intersects a boundary edge.

- At 0 intersections: Outside.
- At 1 intersection: Inside.
- At 2 intersections: Outside.

Consequently, sorting the intersection points along a single scanline and pairing them off ($[x_0, x_1]$, $[x_2, x_3]$, etc.) explicitly defines the interior pixel bounds for that row.

Conceptual Execution Steps:

1. **Determine Spatial Bounds:** Find the minimum and maximum y-coordinates of all vertices (y_{Min} , y_{Max}) to isolate the bounding box of necessary scanlines.
2. **Iterate Scanlines:** For each integer scan line y from y_{Min} to y_{Max} :
 - (a) **Find Intersections:** Mathematically determine the x-coordinates where the horizontal line y crosses each valid polygon edge.
 - (b) **Sort:** Order these intersection x-coordinates from left to right.
 - (c) **Fill Pairs:** Draw horizontal pixel spans between pairs of sequential intersections (Odd index to Even index).

Handling Singularities (Vertices and Horizontal Edges): In order to prevent counting an intersection twice when a scanline passes exactly through a vertex (which would incorrectly invert the inside/outside parity rule), strict rules must be enforced:

- **Horizontal edges are ignored:** They run parallel to the scanline and produce infinitely many or zero meaningful mathematical crossings, contributing nothing to the parity swap.
- **The half-open interval $[y_{Lo}, y_{Hi})$ rule:** The y-range of each edge is treated as inclusive on the bottom and exclusive on the top. This effectively shortens the "upper" edge of a joining vertex intersection by an infinitesimally small fraction. Thus, a shared

vertex joining two non-horizontal edges is counted exactly once by the edge ascending out of it, and zero times by the edge descending into it, preserving the parity chain at geometric corners.

Note: Traditional generalized Scan Line algorithms construct a Global Edge Table (GET) and maintain an Active Edge Table (AET) to optimize $O(N)$ edge comparisons per scanline. In this lab, given the low vertex count of standard basic shapes, we utilize a simplified boundary-box approach that evaluates all edges on every scanline row.

Implementation

```
void ScanLineFill(int x, int y, RGBColor fillColor, RGBColor boundaryColor)
{
    int n = (int)shapeVertices.size();
    if (n < 3) return; // need at least a triangle

    float yMinF = shapeVertices[0].second, yMaxF = shapeVertices[0].second;
    for (auto& v : shapeVertices) {
        if (v.second < yMinF) yMinF = v.second;
        if (v.second > yMaxF) yMaxF = v.second;
    }
    int yMin = (int)std::ceil(yMinF);
    int yMax = (int)std::floor(yMaxF);

    for (int y = yMin; y <= yMax; y++)
    {
        std::vector<float> intersects;

        for (int i = 0; i < n; i++)
        {
            float x0 = shapeVertices[i].first,          y0 = shapeVertices[
i].second;
            float x1 = shapeVertices[(i+1) % n].first,  y1 = shapeVertices
[(i+1) % n].second;

            if (y0 == y1) continue; // skip horizontal edges

            float yLo = std::min(y0, y1), yHi = std::max(y0, y1);
            if ((float)y < yLo || (float)y >= yHi) continue; // half-open
interval
        }
    }
}
```

```

        float xIntersect = x0 + ((float)y - y0) * (x1 - x0) / (y1 - y0);
        intersects.push_back(xIntersect);
    }

    std::sort(intersects.begin(), intersects.end());

    for (int i = 0; i + 1 < (int)intersects.size(); i += 2)
    {
        int xStart = (int)std::ceil(intersects[i]);
        int xEnd   = (int)std::floor(intersects[i + 1]);
        for (int x = xStart; x <= xEnd; x++)
            PutPixel(x, y, fillColor);
    }
}

glFlush();
}

```

Advantages over pixel-propagation algorithms:

Feature	Boundary Fill / Flood Fill	Scan Line
Data source	Pixels in the framebuffer	Vertex list (analytic)
Framebuffer dependency	Yes (reads pixel colors)	None
Speed	$O(n \text{ pixels})$ with large constant (many stack/queue ops)	$O(n \text{ pixels})$ with smaller constant (sequential row writes)
Works without closed pixel boundary	No	Yes
Suitable for curved shapes	Yes (depends on pixel outline)	Yes (approximated by many vertices)

The vertex list reuse: ScanLineFill reads from `shapeVertices`, the same vector that BoundaryFill uses for `SoftDrawLine` and `pointInPolygon`. This single source of truth ensures all three algorithms operate on identical geometry.

5 Unified Fill API

All three fill functions share the same signature:

```
void BoundaryFill (int x, int y, RGBColor fillColor, RGBColor boundaryColor)
    ;
void FloodFill    (int x, int y, RGBColor fillColor, RGBColor boundaryColor)
    ;
void ScanLineFill (int x, int y, RGBColor fillColor, RGBColor boundaryColor)
    ;
```

This makes them interchangeable — swapping one for another in `display()` requires changing only the function name. The calling code in `main.cpp` is:

```
BoundaryFill(fillStartX, fillStartY, savedFillColor, B_Color);
```

To test Flood Fill or Scan Line Fill, replace `BoundaryFill` with `FloodFill` or `ScanLineFill`. All other logic remains identical.

6 Conclusion

This lab demonstrates that a straightforward specification translates into a substantially more complex implementation when deployed on real hardware. The main challenges were:

- **GPU driver incompatibility** with `glReadPixels` — resolved by a CPU-side mirror framebuffer (`softbuf`).
- **Stack overflow** from recursive `BoundaryFill` — resolved by converting to an iterative heap-based stack.
- **Sub-pixel alignment mismatch** between OpenGL rasterizer and Bresenham — resolved by a $\frac{1}{2}$ -pixel projection shift.
- **Redraw lifecycle** of FreeGLUT — resolved by deferring fill to `display()` with saved state.
- **Input safety** (outside-click flood, premature color application) — resolved by point-in-polygon guard and the `colorChosen` flag.

The final program correctly draws all 15 shapes and fills them with any of three colors using all three required algorithms (Boundary Fill, Flood Fill, Scan Line Fill), with robust behavior across all user interaction paths.